AudioRent: Design Pattern Implementation Summary

This document serves as a technical quick-reference for the design patterns implemented during the refactoring of the AudioRent project.

Quick Reference Table

Pattern	Category	Primary File(s)	Impact Area
Factory	Creational	UserFactory.java	User Creation Logic
Singleton	Creational	ApiService.ts	Frontend API Client
Facade	Structural	AuthFacade.java	Authentication Orchestration
Adapter	Structural	GoogleAuthAdapter.java	Third-party SDK Isolation
Decorator	Structural	PriceCalculator.java	Dynamic Rental Pricing
Strategy	Behavioral	UserValidationStrategy.java	Registration Validation
Observer	Behavioral	UserRegisteredEvent.java	Post-Registration Actions

Technical Details & Extension Guide

1. User Creation (Factory)

- **Implementation:** Centralized User instantiation.
- **How to extend:** To add a "Premium" or "Admin" user type, add a new case in UserFactory.java with the required default roles and attributes.

2. API Communication (Singleton)

- **Implementation:** Private constructor in ApiService with a static getInstance() method.
- **Why it matters:** Ensures all frontend requests share the same interceptor logic (e.g.,

attaching Bearer tokens).

3. Auth Orchestration (Facade)

- **Implementation:** AuthFacade aggregates AuthService and JwtUtils.
- **Refactor Note:** Reduced AuthController complexity by 60% by moving multi-step auth logic into the Facade.

4. Third-Party Integration (Adapter)

- **Implementation:** AuthService interacts with an OAuthProvider interface.
- **How to extend:** To support Facebook login, create FacebookAuthAdapter.java implementing OAuthProvider. No changes to AuthService required.

5. Rental Pricing (Decorator)

- **Implementation:** Wraps base rental costs with InsuranceDecorator or ExpressDeliveryDecorator.
- **Why it matters:** Allows users to "stack" features (Insurance + Delivery) without complex nested if statements.

6. Validation Logic (Strategy)

- **Implementation:** List of UserValidationStrategy objects executed in a loop.
- **How to extend:** Add a PasswordStrengthStrategy to the validation list to enforce security requirements without touching the registration flow.

7. Event Handling (Observer)

- **Implementation:** Decoupled event publishing via ApplicationEventPublisher.
- **Current Listeners:** EmailNotificationListener.
- **Future Use:** Easily add a LoyaltyPointsListener to reward new users without modifying the core Auth logic.